

2026 Operating System

Lab 3. EXT2 File System

(Digital Forensic)

2026.05.27

T.A. 위다연

weedayeon@dankook.ac.kr

0. EXT2 File System 실습 목표

- 목표

- 본 과제는 EXT2 파일시스템의 내부 구조를 실제 hex data 기반으로 추적하는 Digital Forensic 형태의 실습이다.
- RAM Disk 위에 생성된 EXT2 파일시스템에서 자신의 학번 끝 세 자리에 따라 할당된 두 개의 Target File을 찾아야 한다. 이를 위해 Superblock, Group Descriptor Table, Inode Table, Directory Entry를 순차적으로 분석하고, Target File의 Inode number와 실제 Data Block 위치를 계산해야 한다.
- 또한 Direct pointer와 Single Indirect Pointer 구조를 분석하여, Target File의 데이터가 어떤 Disk Block에 저장되어 있는지 xxd 결과를 통해 검증해야 한다.

0. EXT2 File System 실습 환경 구성

- 새로운 우분투 이미지를 꼭! 다운로드 받아야 함

- LAB0를 참고하여 실행하되, 이미지 파일은 아래 링크에서 새로 다운로드 할 것

- Window (Virtual Box):

https://drive.google.com/file/d/1VvPzMhofnRewW-FPZOGgfTC7HoQ9jUS-/view?usp=drive_link

- MacOS (UTM):

https://drive.google.com/file/d/1Pc6IVa4V32G9vcNL5rqSc5IT_6_Z8fMt/view?usp=sharing

- Username: User
- Password: 0000

1. File System Layout Analysis

■ Goals of OS Lab3

- EXT2 File System 에서 진행하는 Digital Forensic
- 자신의 학번 끝 세 자리에 따라, 2가지의 파일을 찾아가야 함
 - 자신의 학번이 32222797이라면, 아래 2개의 파일을 찾아야 함
 - Target 1) 7번 디렉터리 안에 97번 파일, Target 2) 7번 디렉터리 안에 79번 파일
 - Target 1은 13번째 Block (인덱스로 12)에 추가 데이터가 기록되어 있으며, Single Indirect Pointer를 통해 접근되는 Block을 분석해야 함
 - Target 2는 100KiB 크기의 Sequential File이며, 12개의 Direct Block과 Single Indirect Pointer를 통해 접근되는 추가 Block들을 분석해야 함

찾아야하는 파일들

Target 1)	18ba0000:	37 2f 39 37 2d 31 33 0a 00 00 00 00 00 00 00 00	7/97-13.....
	18ba0010:	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
	18ba0020:	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
	18ba0030:	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
Target 2)	18bad000:	37 2f 37 39 2d 42 49 47 2d 6c 6f 67 69 63 61 6c	7/79-BIG-logical
	18bad010:	2d 62 6c 6f 63 6b 2d 31 32 0a 2e 2e 2e 2e 2e 2e	-block-12.....
	18bad020:	2e 2e 2e 2e 2e 2e 2e 2e 2e 2e 2e 2e 2e 2e 2e 2e	
	18bad030:	2e 2e 2e 2e 2e 2e 2e 2e 2e 2e 2e 2e 2e 2e 2e 2e	

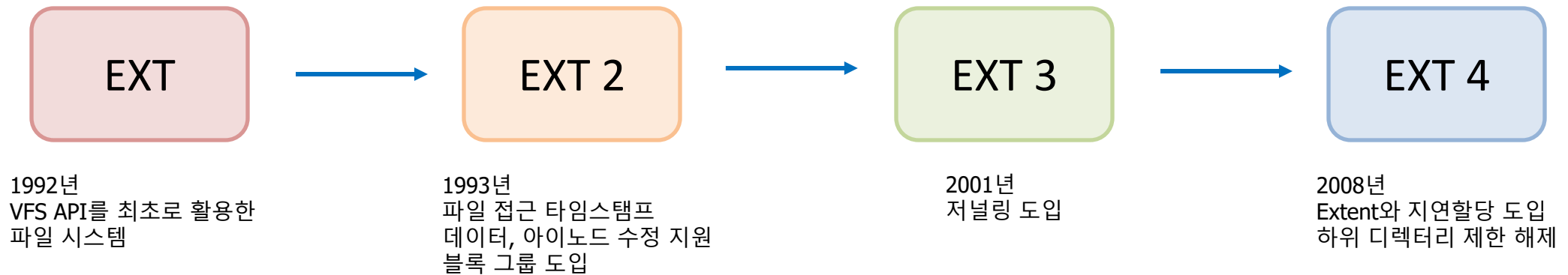
1. File System Layout Analysis

- Goals of OS Lab3 (학번 끝 두 자리가 같은 경우, 중간에 0이 들어가는 경우 필독 !)
 - EXT2 File System 에서 진행하는 Digital Forensic
 - 학번 끝 두 자리가 같은 경우
 - 자신의 학번이 32222133이라면, 아래 2개의 파일을 찾아야 함
 - Target 1) 1번 디렉터리 안에 33번 파일, Target 2) 1번 디렉터리 안에 33번 파일 → 두 Target File이 같아지게 됨
 - 조교가 제공하는 create.sh에서는 Target 2를 $(BC + 1) \bmod 100$ 연산을 통해 처리함
 - 따라서 위의 예제처럼 학번 끝 두 자리가 같은 경우, $(33 + 1) \bmod 100 = 34$ 이므로
 - Target 1) 1번 디렉터리 안에 33번 파일, Target 2) 1번 디렉터리 안에 34번 파일 을 찾으면 됨
 - 학번 끝 세 자리에 0이 들어가는 경우
 - 0번 디렉터리와 00번 파일이 존재함
 - 자신의 학번이 32222102이라면, 아래 2개의 파일을 찾아야 함
 - Target 1) 1번 디렉터리 안에 02번 파일, Target 2) 1번 디렉터리 안에 20번 파일
 - 2번 파일이 아닌, 02번 파일이므로 헛갈리지 않도록 유의

1. File System Layout Analysis

■ EXT2 File System

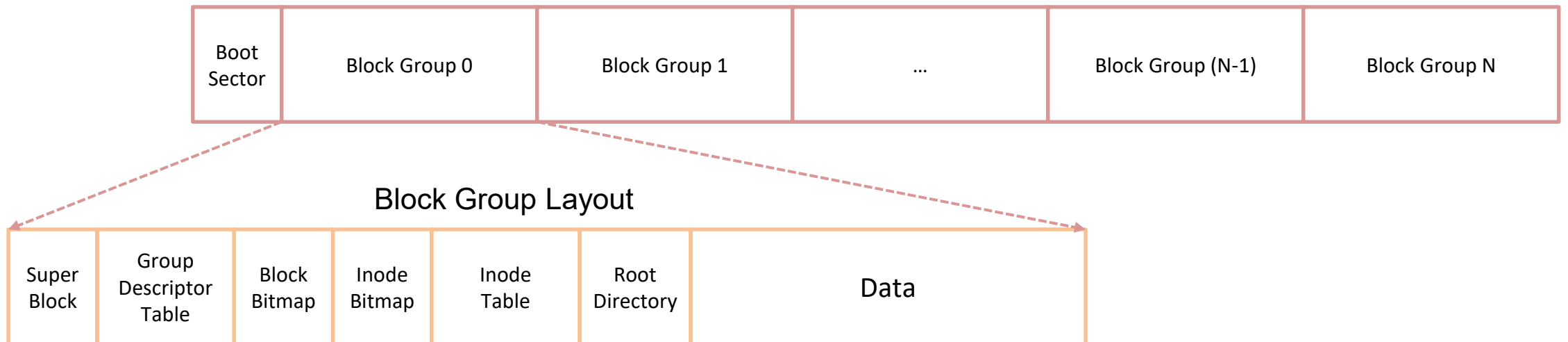
- EXT2(Extended File System, FS)은 유닉스 파일시스템을 개선하여 만들어진 파일시스템이다.
- EXT FS은 리눅스에서 일반 파일 API를 제공하기 위해 가상 파일 시스템 레이어(VFS)가 리눅스 커널에 추가되면서 만들어진 파일시스템이다.
- EXT FS은 1992년 첫 등장 이후에 문제점들을 자체적으로 개선해 나가며 추후에 EXT2, EXT3 그리고 EXT4로 확장되었다.



1. File System Layout Analysis

■ EXT2 FS Layout

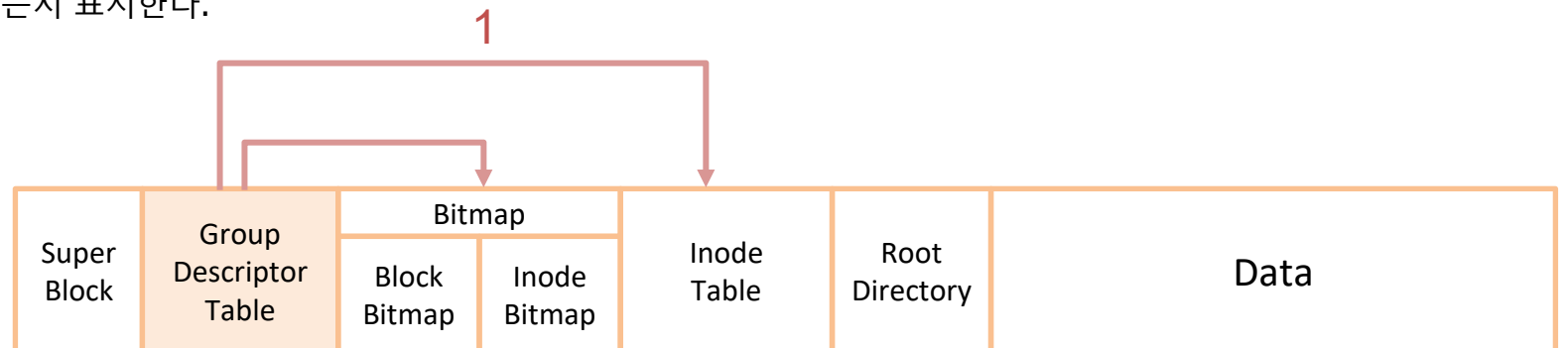
- EXT2는 Boot Sector와 여러 개의 Block Group들로 구성
- Boot Sector
 - 부팅을 위한 여러가지 값을 저장하고 있고, 크기는 2 sectors (1KB = 0x400) 이다.
- Block Group
 - Directory와 같이 논리적으로 연관성이 있는 Data를 한 Block Group에 저장한다.
 - Super Block과 Group Descriptor Table 같은 중요한 영역을 중복해서 관리한다.



1. File System Layout Analysis

■ EXT2 FS Layout

- Super Block
 - FS의 전반적인 정보를 가지고 있다.
 - Block Count, Inode Count, Blocks per Group, Block Group Count ...
 - EXT2 에서 Super Block은 각각의 Block Group에 복제된 상태로 존재한다.
- Group Descriptor Table
 - 각 Block Group에 대한 Description을 나열한 테이블이다.
 - Descriptor에는 Block Bitmap, Inode Bitmap, Inode Table이 포함되어있다.
- Bitmap
 - 해당 Block이나 Inode Number가 할당되었는지 표시한다.



1. File System Layout Analysis

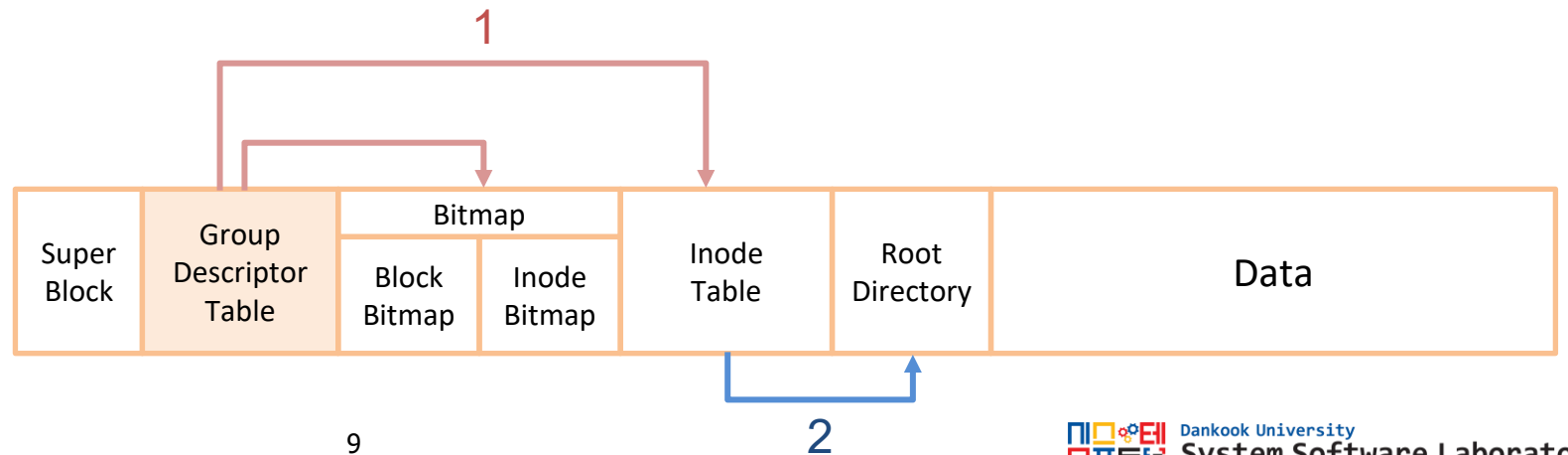
■ EXT2 FS Layout

- Inode Table

- 해당 Block Group에 할당된 Inode들을 나열한 테이블이다.
- Inode는 12개의 Direct Pointer와 3개의 Indirect Pointer로 구성 되어있다.
- 각 Inode의 크기는 256 byte이고 Root Directory의 Inode Number는 2번이다.

- Data

- 파일 혹은 디렉터리가 저장되는 공간이다.
- EXT2에서는 반드시 Inode Table 뒤 영역에 Root Directory가 존재한다.
- 디렉터리는 그에 속한 파일이나 하위 디렉터리의 정보를 담은 Directory Entry의 Table이다.
- 파일은 파일의 내용을 담고있다.



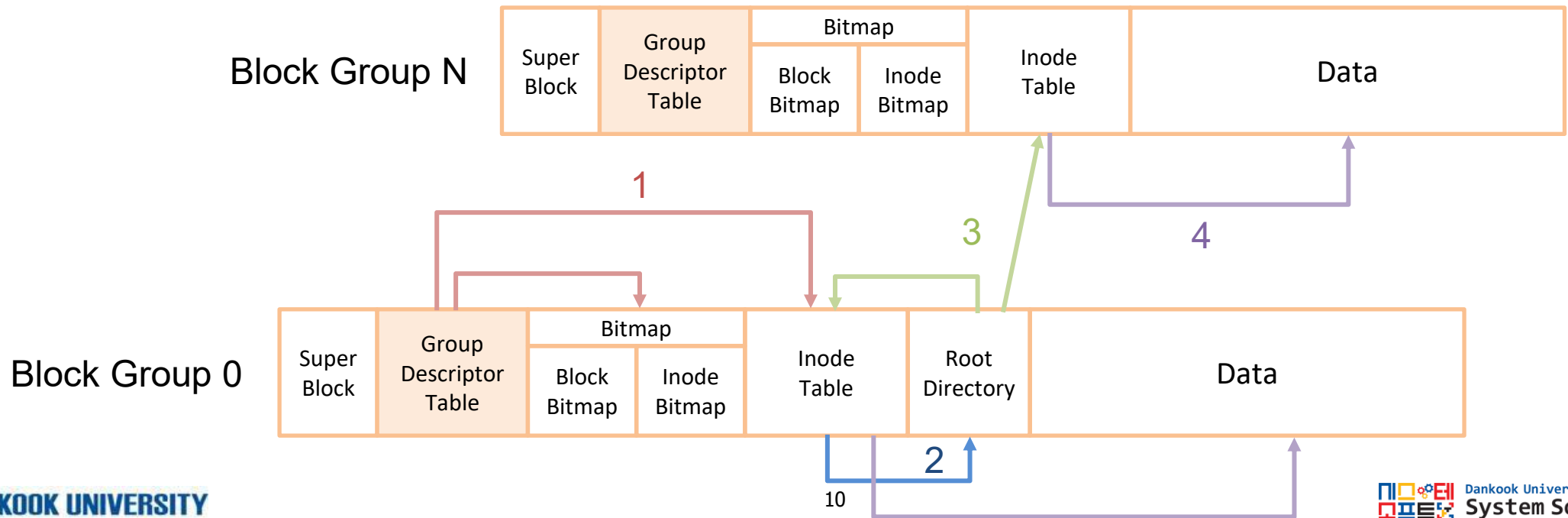
1. File System Layout Analysis

■ EXT2 FS Layout

- Data

• Directory Entry

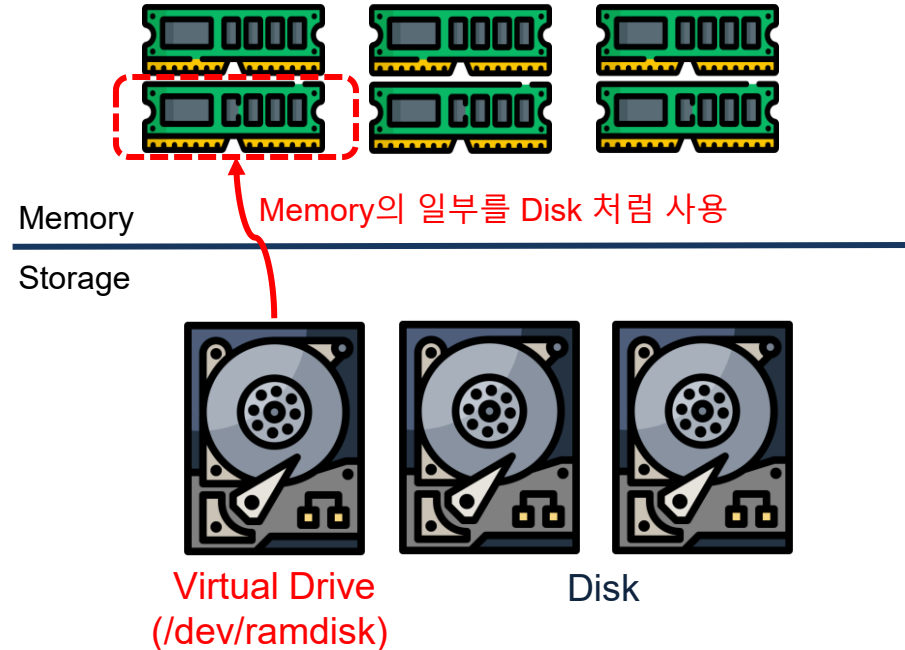
- 하위 디렉터리나 파일의 Inode Number를 가지고 있다.
- Inode Number를 가지고 해당 Group의 Inode Table에서 Inode를 찾아 접근할 수 있다.
- EXT2는 논리적으로 연관성이 있는 Data를 한 Block Group에 저장하려 하지만, 그렇지 못하는 경우도 있다.



1. File System Layout Analysis

■ RAM Disk

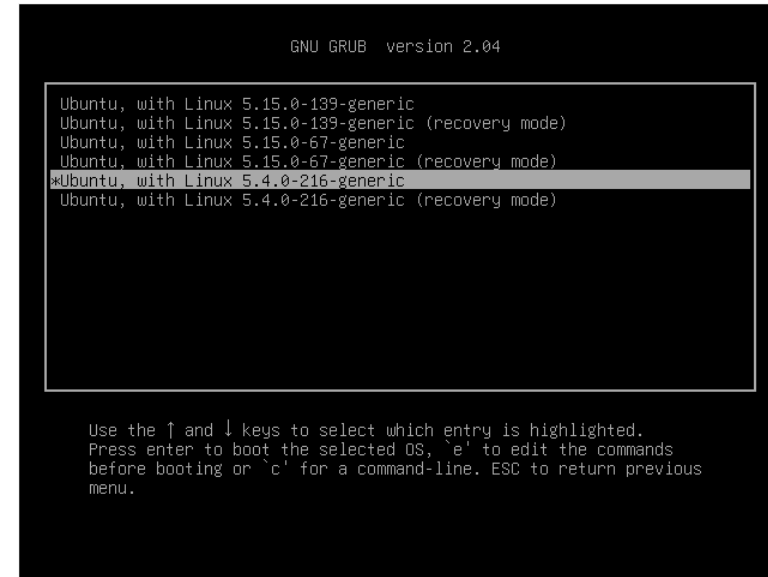
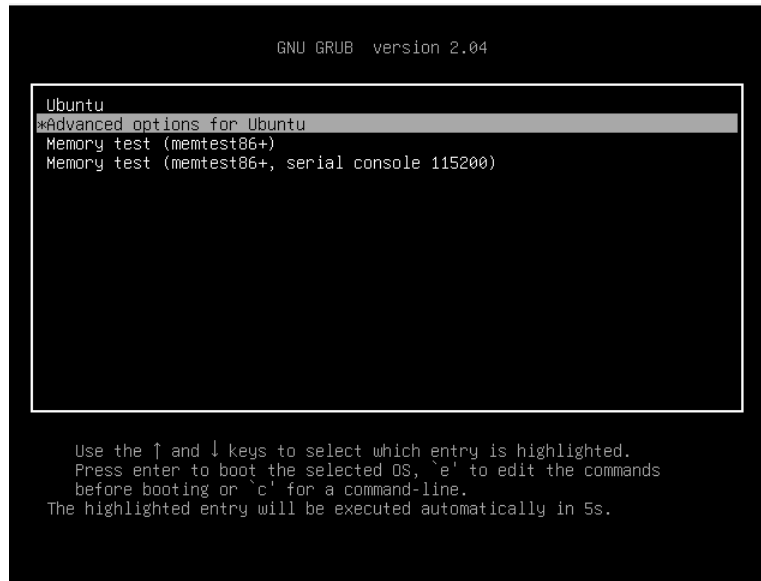
- RAM Disk는 주 기억 장치 활용 저장법이 아닌 RAM(DRAM)을 이용하여 Disk Drive를 구현하는 방식
- 본 과제에서는 실제 Storage를 사용하지 않고, RAM Disk를 생성하여 이를 Storage로 활용한다.
- 과제 자료로 공유된 이미지 파일에는 쉽게 설치할 수 있도록 RAM Disk 설치 파일이 준비되어 있다.
- https://en.wikipedia.org/wiki/RAM_drive



1. File System Layout Analysis

■ 과제 실습 환경 구성

- 3번 슬라이드에서 새로운 이미지 파일을 다운로드 받는다.
- 처음 실행하면 다음과 첫번째 화면이 뜨고, Advanced options for Ubuntu 를 선택한다.
- 선택 후, 커널 버전을 맞추기 위해 **"Ubuntu, with Linux 5.4.0-216-generic"** 을 선택하여 실행한다.



1. File System Layout Analysis

■ RAM Disk

- 2026_DKU_OS_Lab/lab3로 이동
- ls 명령어로 모든 파일이 있는지 확인 후,
sudo su로 root 권한으로 변경
- make
- ramdisk.ko 파일이 있는지 확인
- 모듈 적재 후 확인
- insmod ramdisk.ko
- lsmod | grep ramdisk
- mkdir mnt

```
user@DKU-OS-Lab3:~$ cd 2026_DKU_OS_Lab/lab3/
user@DKU-OS-Lab3:~/2026_DKU_OS_Lab/lab3$ ls
append.c  create.sh  Makefile  os_ext2  ramdisk.c
user@DKU-OS-Lab3:~/2026_DKU_OS_Lab/lab3$ sudo su
[sudo] password for user:
root@DKU-OS-Lab3:/home/user/2026_DKU_OS_Lab/lab3# make
make -C /lib/modules/5.4.0-216-generic/build M=/home/user/2026_DKU_OS_Lab/lab3 modules
make[1]: Entering directory '/usr/src/linux-headers-5.4.0-216-generic'
CC [M] /home/user/2026_DKU_OS_Lab/lab3/ramdisk.o
Building modules, stage 2.
MODPOST 1 modules
CC [M] /home/user/2026_DKU_OS_Lab/lab3/ramdisk.mod.o
LD [M] /home/user/2026_DKU_OS_Lab/lab3/ramdisk.ko
make[1]: Leaving directory '/usr/src/linux-headers-5.4.0-216-generic'
root@DKU-OS-Lab3:/home/user/2026_DKU_OS_Lab/lab3# ls
append.c  Makefile      Module.symvers  ramdisk.c  ramdisk.mod  ramdisk.mod.o
create.sh  modules.order  os_ext2         ramdisk.ko  ramdisk.mod.c  ramdisk.o
```

```
root@DKU-OS-Lab3:/home/user/2026_DKU_OS_Lab/lab3# insmod ramdisk.ko
root@DKU-OS-Lab3:/home/user/2026_DKU_OS_Lab/lab3# lsmod | grep ramdisk
ramdisk                16384  0
```

```
root@DKU-OS-Lab3:/home/user/2026_DKU_OS_Lab/lab3# mkdir mnt
root@DKU-OS-Lab3:/home/user/2026_DKU_OS_Lab/lab3# ls
append.c  Makefile  modules.order  os_ext2  ramdisk.ko  ramdisk.mod.c  ramdisk.o
create.sh  mnt       Module.symvers  ramdisk.c  ramdisk.mod  ramdisk.mod.o
```

※Ramdisk의 특성상 시스템을 종료할 경우 내용이 모두 지워지므로 참고할 것

1. File System Layout Analysis

■ RAM Disk

- 파일시스템 포맷 후 mnt에 마운트
- mkfs.ext2 /dev/ramdisk
- mount /dev/ramdisk ./mnt

```
root@DKU-OS-Lab3:/home/user/2026_DKU_OS_Lab/lab3# mkfs.ext2 /dev/ramdisk
mke2fs 1.45.5 (07-Jan-2020)
Creating filesystem with 131072 4k blocks and 32768 inodes
Filesystem UUID: e00f1367-b0d0-4512-a5e6-ab2e4a7fd37d
Superblock backups stored on blocks:
    32768, 98304

Allocating group tables: done
Writing inode tables: done
Writing superblocks and filesystem accounting information: done

root@DKU-OS-Lab3:/home/user/2026_DKU_OS_Lab/lab3# mount /dev/ramdisk ./mnt
```

- 잘 되었는지 확인
- df -h

```
root@DKU-OS-Lab3:/home/user/2026_DKU_OS_Lab/lab3# df -h
Filesystem      Size  Used Avail Use% Mounted on
udev            956M   0    956M   0% /dev
tmpfs           198M  1.4M   196M   1% /run
/dev/sda5       24G   10G   13G   44% /
tmpfs           986M   0    986M   0% /dev/shm
tmpfs           5.0M  4.0K   5.0M   1% /run/lock
tmpfs           986M   0    986M   0% /sys/fs/cgroup
/dev/loop0      128K  128K   0 100% /snap/bare/5
/dev/loop2      350M  350M   0 100% /snap/gnome-3-38-2004/143
/dev/loop3      347M  347M   0 100% /snap/gnome-3-38-2004/119
/dev/loop1       92M   92M   0 100% /snap/gtk-common-themes/1535
/dev/loop4       50M   50M   0 100% /snap/snapd/18357
/dev/loop5       64M   64M   0 100% /snap/core20/1828
/dev/loop6       46M   46M   0 100% /snap/snap-store/638
/dev/sda1       511M  4.0K   511M   1% /boot/efi
tmpfs           198M  36K   198M   1% /run/user/1000
/dev/sr0         51M   51M   0 100% /media/user/VBox_GAs_7.2.6
/dev/ramdisk    504M  24K   478M   1% /home/user/2026_DKU_OS_Lab/lab3/mnt
```

1. File System Layout Analysis

■ RAM Disk

- create.sh은 학번 끝 세 자리 ABC를 이용하여 target file 1과 target file 2를 자동으로 생성한다.
- ./create.sh 797 (797은 예제이고, 본인 학번 뒤 세자리 입력)
- 실행 시 warning이 뜰 수 있지만 그대로 진행하면 된다.

```
root@DKU-OS-Lab3:/home/user/2026_DKU_OS_Lab/lab3# ./create.sh 797
create files ...
student suffix : 797
target file 1 : /7/97
target file 2 : /7/79
target 1 path : mnt/7/97
target 2 path : mnt/7/79
target 2 size : 102400 bytes
root@DKU-OS-Lab3:/home/user/2026_DKU_OS_Lab/lab3# ls mnt
0 1 2 3 4 5 6 7 8 9 lost+found
```

- 스크립트 실행 후 mnt에 0~9번 디렉터리가 생성된 것을 확인할 수 있다.
- 각 디렉터리 안에는 파일이 0~99번까지 생성 되어있다.

```
root@DKU-OS-Lab3:/home/user/2026_DKU_OS_Lab/lab3# ls mnt/7
00 04 08 12 16 20 24 28 32 36 40 44 48 52 56 60 64 68 72 76 80 84 88 92 96
01 05 09 13 17 21 25 29 33 37 41 45 49 53 57 61 65 69 73 77 81 85 89 93 97
02 06 10 14 18 22 26 30 34 38 42 46 50 54 58 62 66 70 74 78 82 86 90 94 98
03 07 11 15 19 23 27 31 35 39 43 47 51 55 59 63 67 71 75 79 83 87 91 95 99
```

- ls -l mnt/7/97 파일 크기 49160 확인
- ls -l mnt/7/79 파일 크기 102400 확인

```
root@DKU-OS-Lab3:/home/user/2026_DKU_OS_Lab/lab3# ls -l mnt/7/97
-rw-r--r-- 1 root root 49160 5월 6 20:29 mnt/7/97
root@DKU-OS-Lab3:/home/user/2026_DKU_OS_Lab/lab3# ls -l mnt/7/79
-rw-r--r-- 1 root root 102400 5월 6 20:29 mnt/7/79
```

1. File System Layout Analysis

- EXT2 Hex Data 읽는 방법

- 일반 데이터는 빅 엔디안으로 저장 되어있지만, **EXT2의 정보는 리틀 엔디안**으로 저장되어 있다.
- 따라서, EXT2의 Metadata를 읽을 때는 보여지는 데이터를 바이트별로 반대로 읽어야한다.

0x	12	34	56	78
0x	78	56	34	12

- EXT2의 Data Structure와 다른 세부 정보는 아래 사이트에서 확인할 수 있다.
- [The Second Extended File System](#)

1. File System Layout Analysis

- Super Block 영역 분석
- xxd(hexdump) [options] [infile]
 - Infile의 데이터를 16진수 형태로 보여준다.
 - options
 - -l len: len 만큼 데이터를 읽는다.
 - -s [+/-] offset: offset 만큼 offset을 이동한다.
 - +/-: lseek()의 SEEK_SET/SEEK_END와 같음
 - -g bytes: bytes 만큼 묶어서 출력한다.
 - Ex) `xx -g 4 -l 0x100 -s 0x400 /dev/ramdisk`
 - /dev/ramdisk의 0x400 byte부터 0x100 byte 만큼을 4bytes씩 묶어서 보여준다.
 - 추가적인 옵션은 아래 사이트에서 확인할 수 있다.
 - [xxd\(1\): make hexdump/do reverse - Linux man page](#)

```
root@DKU-OS-Lab3:/home/user/2026_DKU_OS_Lab/lab3# xxd -g 4 -l 0x100 -s 0x400 /dev/ramdisk
00000400: 00800000 00000200 99190000 8ff70100 .....
00000410: f57f0000 00000000 02000000 02000000 .....
00000420: 00800000 00800000 00200000 90e8fb69 .....i
00000430: 90e8fb69 0100ffff 53ef0000 01000000 ...i....S.....
00000440: 89e8fb69 00000000 00000000 01000000 ...i.....
00000450: 00000000 0b000000 00010000 38000000 .....8...
00000460: 02000000 03000000 5bbe38e9 502c46bc .....[.8.P,F.
00000470: 85dc84b3 66898e99 00000000 00000000 ....f.....
00000480: 00000000 00000000 2f686f6d 652f7573 ...../home/us
00000490: 65722f32 3032365f 444b555f 4f535f4c er/2026_DKU_OS_L
000004a0: 61622f6c 6162332f 6d6e7400 00000000 ab/lab3/mnt.....
000004b0: 00000000 00000000 00000000 00000000 .....
000004c0: 00000000 00000000 00000000 00001f00 .....
000004d0: 00000000 00000000 00000000 00000000 .....
000004e0: 00000000 00000000 00000000 824668b1 .....Fh.
000004f0: a53946fb 906c84db e4f95161 01000000 .9F..l....Qa....
```

1. File System Layout Analysis

■ Super Block 영역 분석

```
root@DKU-OS-Lab3:/home/user/2026_DKU_OS_Lab/lab3# xxd -g 4 -l 0x100 -s 0x400 /dev/ramdisk
00000400: 00800000 00000200 99190000 8ff70100 .....
00000410: f57f0000 00000000 02000000 02000000 .....
00000420: 00800000 00800000 00200000 90e8fb69 .....i
00000430: 90e8fb69 0100ffff 53ef0000 01000000 ...i...S....
00000440: 89e8fb69 00000000 00000000 01000000 ...i.....
00000450: 00000000 0b000000 00010000 38000000 .....8...
00000460: 02000000 03000000 5bbe38e9 502c46bc .....[.8.P,F.
00000470: 85dc84b3 66898e99 00000000 00000000 ....f.....
00000480: 00000000 00000000 2f686f6d 652f7573 ...../home/us
00000490: 65722f32 3032365f 444b555f 4f535f4c er/2026_DKU_OS_L
000004a0: 61622f6c 6162332f 6d6e7400 00000000 ab/lab3/mnt....
000004b0: 00000000 00000000 00000000 00000000 .....
000004c0: 00000000 00000000 00000000 00001f00 .....
000004d0: 00000000 00000000 00000000 00000000 .....
000004e0: 00000000 00000000 00000000 824668b1 .....Fh.
000004f0: a53946fb 906c84db e4f95161 01000000 .9F..l....0a....
```

linux/fs/ext2/ext2.h

```
struct ext2_super_block {
    __le32 s_inodes_count; /* Inodes count */
    __le32 s_blocks_count; /* Blocks count */
    __le32 s_r_blocks_count; /* Reserved blocks count */
    __le32 s_free_blocks_count; /* Free blocks count */
    __le32 s_free_inodes_count; /* Free inodes count */
    __le32 s_first_data_block; /* First Data Block */
    __le32 s_log_block_size; /* Block size */
    __le32 s_log_frag_size; /* Fragment size */
    __le32 s_blocks_per_group; /* # Blocks per group */
    __le32 s_frags_per_group; /* # Fragments per group */
    __le32 s_inodes_per_group; /* # Inodes per group */
    __le32 s_mtime; /* Mount time */
    __le32 s_wtime; /* Write time */
}
```

	00	01	02	03	04	05	06	07	08	09	0a	0b	0c	0d	e0	0f
00	inode count				block count				res block count				free block count			
10	free inode count				first data block				log block size				log frag size			
20	block per group				frag per group				inode per group				mtime			
30	wtime				mount count		max mount size		magic		state		errors		minor version	
40	last check				check interval				creator OS				major version			
50	def_res uid		def_res gid		first non-reserved inode				inode size		block group number		compatible feature flag			
60	incompatible feature flag				feature read only compat				uuid (16 byte)							
70									volume name (16 byte)							
80																
90																
a0	last mounted (64 byte)								prealloc dir block							
b0									prealloc block							
c0																
d0									algorithm usage bitmap							
e0	journal inode number				journal device				last orphan							
f0	hash seed (16 byte)													pad	padding	
100	default mount option				first meta block				default hash version							

1. File System Layout Analysis

■ Super Block 영역 분석

```

root@DKU-OS-Lab3:/home/user/2026_DKU_OS_Lab/lab3# xxd -g 4 -l 0x100 -s 0x400 /dev/ramdisk
00000400: 00800000 00000200 99190000 8ff70100 .....
00000410: f57f0000 00000000 02000000 02000000 .....
00000420: 00800000 00800000 00200000 90e8fb69 .....i
00000430: 90e8fb69 0100ffff 53ef0000 01000000 ...i....S.....
00000440: 89e8fb69 00000000 00000000 01000000 ...i.....
00000450: 00000000 0b000000 00000000 38000000 .....8...
00000460: 02000000 03000000 5bbe38e9 502c46bc .....[.8.P,F.
00000470: 85dc84b3 66898e99 00000000 00000000 ....f.....
00000480: 00000000 00000000 2f686f6d 652f7573 ...../home/us
00000490: 65722f32 3032365f 444b555f 4f535f4c er/2026_DKU_OS_L
000004a0: 61622f6c 6162332f 6d6e7400 00000000 ab/lab3/mnt....
000004b0: 00000000 00000000 00000000 00000000 .....
000004c0: 00000000 00000000 00000000 00001f00 .....
000004d0: 00000000 00000000 00000000 00000000 .....
000004e0: 00000000 00000000 00000000 824668b1 .....Fh.
000004f0: a53946fb 906c84db e4f95161 01000000 .9F..l....Qa....
    
```

- Inode count: 0x8000
- Block count: 0x20000
- Log block size: 0x2
- Block per group: 0x8000
- Inodes per group: 0x2000
- Block group number: 0x0

	00	01	02	03	04	05	06	07	08	09	0a	0b	0c	0d	e0	0f
00	inode count				block count				res block count				free block count			
10	free inode count				first data block				log block size				log frag size			
20	block per group				frag per group				inode per group				mtime			
30	wtime				mount count		max mount size		magic		state		errors		minor version	
40	last check				check interval				creator OS				major version			
50	def_res uid		def_res gid		first non-reserved inode				inode size		block group number		compatible feature flag			

1. File System Layout Analysis

■ Group Descriptor Table 영역 분석

- 첫번째 Group Descriptor Table은 RAM Disk의 1블록 이후부터 시작 (1블록 = 4KB = 0x1000)

```
root@DKU-OS-Lab3:/home/user/2026_DKU_OS_Lab/lab3# xxd -g 4 -l 0x100 -s 0x1000 /dev/ramdisk
00001000: 21000000 22000000 23000000 cb7dc61e !..."...#....}..
00001010: 05000400 00000000 00000000 00000000 .....
00001020: 21800000 22800000 23800000 da7dd11e !..."...#....}..
00001030: 03000400 00000000 00000000 00000000 .....
00001040: 00000100 01000100 02000100 fc75361f .....u6.
00001050: 02000400 00000000 00000000 00000000 .....
00001060: 21800100 22800100 23800100 2c76361f !..."...#...,v6.
00001070: 02000400 00000000 00000000 00000000 .....
00001080: 00000000 00000000 00000000 00000000 .....
00001090: 00000000 00000000 00000000 00000000 .....
000010a0: 00000000 00000000 00000000 00000000 .....
000010b0: 00000000 00000000 00000000 00000000 .....
000010c0: 00000000 00000000 00000000 00000000 .....
000010d0: 00000000 00000000 00000000 00000000 .....
000010e0: 00000000 00000000 00000000 00000000 .....
000010f0: 00000000 00000000 00000000 00000000 .....
```

linux/fs/ext2/ext2.h

```
struct ext2_group_desc
{
    __le32 bg_block_bitmap;          /* Blocks bitmap block */
    __le32 bg_inode_bitmap;          /* Inodes bitmap block */
    __le32 bg_inode_table;           /* Inodes table block */
    __le16 bg_free_blocks_count;      /* Free blocks count */
    __le16 bg_free_inodes_count;      /* Free inodes count */
    __le16 bg_used_dirs_count;        /* Directories count */
    __le16 bg_pad;
    __le32 bg_reserved[3];
};
```

- Group 0

- Block bitmap: 0x21 블록부터 시작
- Inode bitmap: 0x22 블록부터 시작
- Inode table: 0x23 블록부터 시작
- Inode가 속한 그룹은 (Inode Number -1) / Block per Group 번 Block Group이다.
- Ext2에서 Root Inode Number는 2번이다.

	00	01	02	03	04	05	06	07	08	09	0a	0b	0c	0d	e0	0f
00	block bitmap				inode bitmap				inode table				free blk cnt		free ino cnt	
01	used dir cnt		padding		reserved (padding)											

1. File System Layout Analysis

■ Group Descriptor Table 영역 분석

- 첫번째 Group Descriptor Table은 RAM Disk의 1블록 이후부터 시작 (1블록 = 4KB = 0x1000)

```
root@DKU-OS-Lab3:/home/user/2026_DKU_OS_Lab/lab3# xxd -g 4 -l 0x100 -s 0x1000 /dev/ramdisk
00001000: 21000000 22000000 23000000 cb7dc61e !..."...#....}..
00001010: 05000400 00000000 00000000 00000000 .....
00001020: 21800000 22800000 23800000 da7dd11e !..."...#....}..
00001030: 03000400 00000000 00000000 00000000 .....
00001040: 00000100 01000100 02000100 fc75361f .....u6.
00001050: 02000400 00000000 00000000 00000000 .....
00001060: 21800100 22800100 23800100 2c76361f !..."...#....,v6.
00001070: 02000400 00000000 00000000 00000000 .....
00001080: 00000000 00000000 00000000 00000000 .....
00001090: 00000000 00000000 00000000 00000000 .....
000010a0: 00000000 00000000 00000000 00000000 .....
000010b0: 00000000 00000000 00000000 00000000 .....
000010c0: 00000000 00000000 00000000 00000000 .....
000010d0: 00000000 00000000 00000000 00000000 .....
000010e0: 00000000 00000000 00000000 00000000 .....
000010f0: 00000000 00000000 00000000 00000000 .....
```

linux/fs/ext2/ext2.h

```
struct ext2_group_desc
{
    __le32 bg_block_bitmap;          /* Blocks bitmap block */
    __le32 bg_inode_bitmap;          /* Inodes bitmap block */
    __le32 bg_inode_table;           /* Inodes table block */
    __le16 bg_free_blocks_count;      /* Free blocks count */
    __le16 bg_free_inodes_count;      /* Free inodes count */
    __le16 bg_used_dirs_count;        /* Directories count */
    __le16 bg_pad;
    __le32 bg_reserved[3];
};
```

- Inode가 속한 그룹은 (Inode Number -1) / Block per Group 번 Block Group이다.
- Ext2에서 Root Inode Number는 2번이다.
- 예시에서 Inodes per Group은 0x2000 이므로
 - Root's Block Group: $(2-1) / 0x2000 = 0$
 - Root's Index: $(2-1) \% 0x2000 = 1$
 - 0번 Block Group의 Inode Table의 1번째에 위치한다.

	00	01	02	03	04	05	06	07	08	09	0a	0b	0c	0d	e0	0f
00	block bitmap				inode bitmap				inode table				free blk cnt		free ino cnt	
01	used dir cnt		padding		reserved (padding)											

1. File System Layout Analysis

■ Group Descriptor Table 영역 분석

- Inode의 크기는 0x100 byte 이다.
(inode + padding)
- Root가 속한 Block Group은 0번이고,
Index 는 1이다.
- 따라서 0번 Block Group의 Inode Table의
0x100부터가 Root Inode 이다.

```
root@DKU-OS-Lab3:/home/user/2026_DKU_OS_Lab/lab3# xxd -g 4 -l 0x1000 -s 0x23000 /dev/ramdisk
00023000: 00000000 00000000 89e8fb69 89e8fb69 .....i...i
00023010: 89e8fb69 00000000 00000000 00000000 ...i.....
00023020: 00000000 00000000 00000000 00000000 .....
00023030: 00000000 00000000 00000000 00000000 .....
00023040: 00000000 00000000 00000000 00000000 .....
00023050: 00000000 00000000 00000000 00000000 .....
00023060: 00000000 00000000 00000000 00000000 .....
00023070: 00000000 00000000 00000000 00000000 .....
00023080: 00000000 00000000 00000000 00000000 .....
00023090: 00000000 00000000 00000000 00000000 .....
000230a0: 00000000 00000000 00000000 00000000 .....
000230b0: 00000000 00000000 00000000 00000000 .....
000230c0: 00000000 00000000 00000000 00000000 .....
000230d0: 00000000 00000000 00000000 00000000 .....
000230e0: 00000000 00000000 00000000 00000000 .....
000230f0: 00000000 00000000 00000000 00000000 .....
00023100: ed410000 00100000 90e8fb69 a8e8fb69 .A.....i...i
00023110: a8e8fb69 00000000 00000d00 08000000 ...i.....
00023120: 00000000 0a000000 23020000 00000000 .....#.
00023130: 00000000 00000000 00000000 00000000 .....
00023140: 00000000 00000000 00000000 00000000 .....
00023150: 00000000 00000000 00000000 00000000 .....
00023160: 00000000 00000000 00000000 00000000 .....
00023170: 00000000 00000000 00000000 00000000 .....
00023180: 20000000 8830ca0e 8830ca0e 606fcc05 ....0...0..`o..
00023190: 89e8fb69 00000000 00000000 00000000 ...i.....
000231a0: 00000000 00000000 00000000 00000000 .....
000231b0: 00000000 00000000 00000000 00000000 .....
000231c0: 00000000 00000000 00000000 00000000 .....
000231d0: 00000000 00000000 00000000 00000000 .....
000231e0: 00000000 00000000 00000000 00000000 .....
000231f0: 00000000 00000000 00000000 00000000 .....
```

1. File System Layout Analysis

■ Inode Table 영역 분석

```

00023100: ed410000 00100000 90e8fb69 a8e8fb69 .A.....i..i
00023110: a8e8fb69 00000000 00000d00 08000000 ...i.....
00023120: 00000000 0a000000 23020000 00000000 .....#.....
00023130: 00000000 00000000 00000000 00000000 .....
00023140: 00000000 00000000 00000000 00000000 .....
00023150: 00000000 00000000 00000000 00000000 .....
00023160: 00000000 00000000 00000000 00000000 .....
00023170: 00000000 00000000 00000000 00000000 .....
00023180: 20000000 8830ca0e 8830ca0e 606fcc05 ....0...0..`o..
00023190: 89e8fb69 00000000 00000000 00000000 ...i.....
000231a0: 00000000 00000000 00000000 00000000 .....
000231b0: 00000000 00000000 00000000 00000000 .....
000231c0: 00000000 00000000 00000000 00000000 .....
000231d0: 00000000 00000000 00000000 00000000 .....
000231e0: 00000000 00000000 00000000 00000000 .....
000231f0: 00000000 00000000 00000000 00000000 .....
    
```

- Mode: 0x41ed = drwxr-xr-x

Constant	Value	Description
-- file format --		
EXT2_S_IFSOCK	0xC000	socket
EXT2_S_IFLNK	0xA000	symbolic link
EXT2_S_IFREG	0x8000	regular file
EXT2_S_IFBLK	0x6000	block device
EXT2_S_IFDIR	0x4000	directory
EXT2_S_IFCHR	0x2000	character device
EXT2_S_IFIFO	0x1000	fifo
-- process execution user/group override --		
EXT2_S_ISUID	0x0800	Set process User ID
EXT2_S_ISGID	0x0400	Set process Group ID
EXT2_S_ISVTX	0x0200	sticky bit
-- access rights --		
EXT2_S_IRUSR	0x0100	user read
EXT2_S_IWUSR	0x0080	user write
EXT2_S_IXUSR	0x0040	user execute
EXT2_S_IRGRP	0x0020	group read
EXT2_S_IWGRP	0x0010	group write
EXT2_S_IXGRP	0x0008	group execute
EXT2_S_IROTH	0x0004	others read
EXT2_S_IWOTH	0x0002	others write
EXT2_S_IXOTH	0x0001	others execute

- Block pointer 0: 0x223 block

linux/fs/ext2/ext2.h

```

struct ext2_inode {
    __le16 i_mode;        /* File mode */
    __le16 i_uid;         /* Low 16 bits of Owner Uid */
    __le32 i_size;        /* Size in bytes */
    __le32 i_atime;       /* Access time */
    __le32 i_ctime;       /* Creation time */
    __le32 i_mtime;       /* Modification time */
    __le32 i_dtime;       /* Deletion Time */
    __le16 i_gid;         /* Low 16 bits of Group Id */
    __le16 i_links_count; /* Links count */
    __le32 i_blocks;      /* Blocks count */
    __le32 i_flags;        /* File flags */
    union {
        struct {
            __le32 l_i_reserved1;
        } linux1;
        struct {
            __le32 h_i_translator;
        } hurd1;
        struct {
            __le32 m_i_reserved1;
        } masix1;
    } osd1;                /* OS dependent 1 */
}
    
```

	00	01	02	03	04	05	06	07	08	09	0a	0b	0c	0d	e0	0f
00	mode		uid		size				access time				change time			
10	modification time				deletion time				gid		link count		blocks			
20	flags				OS description 1											
30																
40																
50																
60																
60					generation				file access control list				dir access control list			
70	fragmentation blk addr				OS description 2											

1. File System Layout Analysis

■ Data 영역 분석

```
root@DKU-OS-Lab3:/home/user/2026_DKU_OS_Lab/lab3# xxd -g 4 -l 0x1000 -s 0x223000 /dev/ramdisk
00223000: 02000000 0c000102 2e000000 02000000 .....
00223010: 0c000202 2e2e0000 0b000000 14000a02 .....
00223020: 6c6f7374 2b666f75 6e640000 01400000 lost+found...@..
00223030: 0c000102 30000000 01200000 0c000102 ....0....
00223040: 31000000 01600000 0c000102 32000000 1....`.....2...
00223050: 0c000000 0c000102 33000000 71000000 .....3...q...
00223060: 0c000102 34000000 66600000 0c000102 ....4...f`.....
00223070: 35000000 66200000 0c000102 36000000 5...f .....6...
00223080: 66400000 0c000102 37000000 cb200000 f@.....7....
00223090: 0c000102 38000000 d6000000 680f0102 ....8.....h...
002230a0: 39000000 00000000 00000000 00000000 9.....
```

- Data

- Inode Number: 0x4066
- File Type: 0x2 = Directory
- 속한 Block Group: $(0x4066-1) / 2000 = 2$ 번 Block Group
- Inode Table Index: $(0x4066-1) \% 2000 = 101$ 번 Index
- 7번 디렉터리의 Inode는 2번 Block Group의 Inode Table에서 101번째에 위치함을 알 수 있다.

Constant Name	Value	Description
EXT2_FT_UNKNOWN	0	Unknown File Type
EXT2_FT_REG_FILE	1	Regular File
EXT2_FT_DIR	2	Directory File
EXT2_FT_CHRDEV	3	Character Device
EXT2_FT_BLKDEV	4	Block Device
EXT2_FT_FIFO	5	Buffer File
EXT2_FT SOCK	6	Socket File
EXT2_FT_SYMLINK	7	Symbolic Link

	00	01	02	03	04	05	06	07	08	09	0a	0b	0c	0d	e0	0f
00	inode				record len		name len	file type	name (~255 byte)							

1. File System Layout Analysis

- 파일 찾기

- 파일 찾기 순서는 다음과 같다.
 - Root Directory에서 찾을 File이 속한 Directory의 Inode Number를 찾는다.
 - Directory가 속한 Block Group의 Inode Table에서 Inode를 찾는다.
 - 찾은 Directory Entry에서 File의 Inode Number를 찾는다.
 - File이 속한 Block Group의 Inode Table에서 Inode를 찾는다.
- 이전의 예시들과 같이 파일을 찾아가는 과정을 세세히 보여야한다.

Lab3. EXT2 File System Bonus 환경 구성

■ EXT2 File System Mount, Lookup 시 자신의 학번 및 이름 출력 (Bonus 과제는 선택)

- 첫 번째 실습에서 mnt를 사용하고 있기 때문에 이를 정리해야 한다.

- umount /dev/ramdisk
- rmmod ramdisk
- lsmod | grep ramdisk (모듈이 없어졌는지 확인)

```
root@DKU-OS-Lab3:/home/user/2026_DKU_OS_Lab/lab3# umount /dev/ramdisk
root@DKU-OS-Lab3:/home/user/2026_DKU_OS_Lab/lab3# rmmod ramdisk
root@DKU-OS-Lab3:/home/user/2026_DKU_OS_Lab/lab3# lsmod | grep ramdisk
```

- insmod ramdisk.ko
- lsmod | grep ramdisk

```
root@DKU-OS-Lab3:/home/user/2026_DKU_OS_Lab/lab3# insmod ramdisk.ko
root@DKU-OS-Lab3:/home/user/2026_DKU_OS_Lab/lab3# lsmod | grep ramdisk
ramdisk                16384  0
```

- cd os_ext2
- ls

```
root@DKU-OS-Lab3:/home/user/2026_DKU_OS_Lab/lab3# cd os_ext2/
root@DKU-OS-Lab3:/home/user/2026_DKU_OS_Lab/lab3/os_ext2# ls
acl.c      dir.c      file.o      inode.o      Makefile      namei.o      os_ext2.mod.o  symlink.c  xattr.h
acl.h      dir.o      ialloc.c    ioctl.c      modules.order  os_ext2.ko    os_ext2.o      symlink.o  xattr_security.c
balloc.c   ext2.h     ialloc.o    ioctl.o      Module.symvers  os_ext2.mod  super.c        tags       xattr_trusted.c
balloc.o   file.c     inode.c     Kconfig      namei.c        os_ext2.mod.c  super.o        xattr.c    xattr_user.c
```

- 최종 목표: 소스를 수정하여 마운트 시 자신의 이름이 출력되면 성공
- 출력 문구: **“os_ext2: (학번) (이름) OS Lab3”**

```
root@DKU-OS-Lab3:/home/user/2026_DKU_OS_Lab/lab3# dmesg | grep os_ext2
[ 9408.193236] os_ext2 : 32222797 WeeDayeon OS Lab3
```

2. File System Modification

■ Super Block

- 마운트 된 File System에 대한 정보를 가진 객체
- File System Control Block에 대응한다.

linux/include/linux/super_types.h

```
struct super_block {
    struct list_head      s_list;          /* Keep this first */
    dev_t                 s_dev;           /* search index; _not_ kdev_t */
    unsigned char         s_blocksize_bits;
    unsigned long         s_blocksize;
    loff_t                s_maxbytes;      /* Max file size */
    struct file_system_type *s_type;
    const struct super_operations *s_op;
    const struct dquot_operations *dq_op;
    const struct quotactl_ops *s_qcop;
    const struct export_operations *s_export_op;
    unsigned long         s_flags;
    unsigned long         s_iflags;       /* internal SB_I_* flags */
    unsigned long         s_magic;
    struct dentry          *s_root;
    struct rw_semaphore    s_umount;
    int                   s_count;
    atomic_t              s_active;
#ifdef CONFIG_SECURITY
    void                  *s_security;
#endif
    const struct xattr_handler *const *s_xattr;
#ifdef CONFIG_FS_ENCRYPTION
    const struct fscrypt_operations *s_cop;
    struct fscrypt_keyring *s_master_keys; /* master crypto keys in use */
#endif
#ifdef CONFIG_FS_VERITY
    const struct fsverity_operations *s_vop;
#endif
#ifdef IS_ENABLED(CONFIG_UNICODE)
    struct unicode_map     *s_encoding;
    __u16                  s_encoding_flags;
#endif
    struct hlist_bl_head  s_roots;         /* alternate root dentries for NFS */
    struct mount           *s_mounts;      /* list of mounts; _not_ for fs use */
    struct block_device   *s_bdev;        /* can go away once we use an accessor for @s_bdev_file */
    struct file            *s_bdev_file;
    struct backing_dev_info *s_bdi;
    struct mtd_info        *s_mtd;
    struct hlist_node      s_instances;
    unsigned int           s_quota_types; /* Bitmask of supported quota types */
    struct quota_info      s_dquot;       /* Diskquota specific options */

    struct sb_writers      s_writers;
}
```

2. File System Modification

- Super Block

- Super Block의 초기화는 File System이 mount 되는 시점에 수행된다.

```
static struct file_system_type ext2_fs_type = {
    .owner          = THIS_MODULE,
    .name           = "os_ext2",
    .mount           = ext2_mount,
    .kill_sb         = kill_block_super,
    .fs_flags        = FS_REQUIRES_DEV,
};
MODULE_ALIAS_FS("ext2");
```

```
static struct dentry *ext2_mount(struct file_system_type *fs_type,
    int flags, const char *dev_name, void *data)
{
    printk(KERN_ERR ""); //os_ext2 : Your Name OS Lab3
    return mount_bdev(fs_type, flags, dev_name, data, ext2_fill_super);
}
```

```
static int ext2_fill_super(struct super_block *sb, void *data, int silent)
{
    struct dax_device *dax_dev = fs_dax_get_by_bdev(sb->s_bdev);
    struct buffer_head * bh;
    struct ext2_sb_info * sbi;
    struct ext2_super_block * es;
    struct inode *root;
    unsigned long block;
    unsigned long sb_block = get_sb_block(&data);
    unsigned long logic_sb_block;
    unsigned long offset = 0;
    unsigned long def_mount_opts;
    long ret = -ENOMEM;
    int blocksize = BLOCK_SIZE;
    int db_count;
    int i, j;
    __le32 features;
    int err;
    struct ext2_mount_options opts;
```

2. File System Modification

■ Super Block

- ext2_fill_super 함수에서는 Disk의 Super Block을 읽고 ext2_boot_sector(boot record), superblock(ext2_sb_info), root directory(inode) 등의 정보를 초기화한다.

```
set_opt(opts.s_mount_opt, RESERVATION);

if (!parse_options((char *) data, sb, &opts))
    goto failed_mount;

sbi->s_mount_opt = opts.s_mount_opt;
sbi->s_resuid = opts.s_resuid;
sbi->s_resgid = opts.s_resgid;

sb->s_flags = (sb->s_flags & ~SB_POSIXACL) |
    ((EXT2_SB(sb)->s_mount_opt & EXT2_MOUNT_POSIX_ACL) ?
    SB_POSIXACL : 0);
sb->s_iflags |= SB_I_CGROUPWB;
```

superblock 초기화

```
if (blocksize != BLOCK_SIZE) {
    logic_sb_block = (sb_block * BLOCK_SIZE) / blocksize;
    offset = (sb_block * BLOCK_SIZE) % blocksize;
} else {
    logic_sb_block = sb_block;
}

if (!(bh = sb_bread(sb, logic_sb_block))) {
    ext2_msg(sb, KERN_ERR, "error: unable to read superblock");
    goto failed_sbi;
}
```

superblock 읽기

```
sbi->s_frag_size = EXT2_MIN_FRAG_SIZE <<
    le32_to_cpu(es->s_log_frag_size);
if (sbi->s_frag_size == 0)
    goto cantfind_ext2;
sbi->s_frags_per_block = sb->s_blocksize / sbi->s_frag_size;

sbi->s_blocks_per_group = le32_to_cpu(es->s_blocks_per_group);
sbi->s_frags_per_group = le32_to_cpu(es->s_frags_per_group);
sbi->s_inodes_per_group = le32_to_cpu(es->s_inodes_per_group);

sbi->s_inodes_per_block = sb->s_blocksize / EXT2_INODE_SIZE(sb);
```

ext2_sb_info 초기화

```
root = ext2_iget(sb, EXT2_ROOT_INO);
if (IS_ERR(root)) {
    ret = PTR_ERR(root);
    goto failed_mount3;
}
if (!IS_ISDIR(root->i_mode) || !root->i_blocks || !root->i_size) {
    iput(root);
    ext2_msg(sb, KERN_ERR, "error: corrupt root inode, run e2fsck");
    goto failed_mount3;
}

sb->s_root = d_make_root(root);
```

root 생성

2. File System Modification

- EXT2 File System Mount, Lookup 시 자신의 학번 및 이름 출력

- 소스 코드 수정 후 make

```
root@DKU-OS-Lab3:/home/user/2026_DKU_OS_Lab/lab3/os_ext2# make
make -C /lib/modules/5.4.0-216-generic/build M=/home/user/2026_DKU_OS_Lab/lab3/os_ext2 modules
make[1]: Entering directory '/usr/src/linux-headers-5.4.0-216-generic'
CC [M] /home/user/2026_DKU_OS_Lab/lab3/os_ext2/balloc.o
CC [M] /home/user/2026_DKU_OS_Lab/lab3/os_ext2/dir.o
CC [M] /home/user/2026_DKU_OS_Lab/lab3/os_ext2/file.o
CC [M] /home/user/2026_DKU_OS_Lab/lab3/os_ext2/ialloc.o
CC [M] /home/user/2026_DKU_OS_Lab/lab3/os_ext2/inode.o
CC [M] /home/user/2026_DKU_OS_Lab/lab3/os_ext2/ioctl.o
CC [M] /home/user/2026_DKU_OS_Lab/lab3/os_ext2/namei.o
CC [M] /home/user/2026_DKU_OS_Lab/lab3/os_ext2/super.o
CC [M] /home/user/2026_DKU_OS_Lab/lab3/os_ext2/symlink.o
LD [M] /home/user/2026_DKU_OS_Lab/lab3/os_ext2/os_ext2.o
Building modules, stage 2.
MODPOST 1 modules
CC [M] /home/user/2026_DKU_OS_Lab/lab3/os_ext2/os_ext2.mod.o
LD [M] /home/user/2026_DKU_OS_Lab/lab3/os_ext2/os_ext2.ko
make[1]: Leaving directory '/usr/src/linux-headers-5.4.0-216-generic'
```

- insmod os_ext2.ko
- lsmod | grep os_ext2 (잘 적재되었는지 확인)
- cd ..

```
root@DKU-OS-Lab3:/home/user/2026_DKU_OS_Lab/lab3/os_ext2# insmod os_ext2.ko
root@DKU-OS-Lab3:/home/user/2026_DKU_OS_Lab/lab3/os_ext2# lsmod | grep os_ext2
os_ext2                73728  0
root@DKU-OS-Lab3:/home/user/2026_DKU_OS_Lab/lab3/os_ext2# cd ..
```

2. File System Modification

- EXT2 File System Mount, Lookup 시 자신의 학번 및 이름 출력

- ext2로 포맷 후 os_ext로 mount
- mkfs.ext2 /dev/ramdisk
- mount -t os_ext2 /dev/ramdisk ./mnt

```
root@DKU-OS-Lab3:/home/user/2026_DKU_OS_Lab/lab3# mkfs.ext2 /dev/ramdisk
mke2fs 1.45.5 (07-Jan-2020)
Creating filesystem with 131072 4k blocks and 32768 inodes
Filesystem UUID: 96e3096a-0baf-416a-a987-66a14ec75e92
Superblock backups stored on blocks:
    32768, 98304

Allocating group tables: done
Writing inode tables: done
Writing superblocks and filesystem accounting information: done

root@DKU-OS-Lab3:/home/user/2026_DKU_OS_Lab/lab3# mount -t os_ext2 /dev/ramdisk ./mnt
```

- dmesg | grep os_ext2

```
root@DKU-OS-Lab3:/home/user/2026_DKU_OS_Lab/lab3# dmesg | grep os_ext2
[ 9408.193236] os_ext2 : 32222797 WeeDayeon OS Lab3
```

- 위와 같이 "**os_ext2: (학번) (이름) OS Lab3**" 문구가 출력되었다면 성공

3. Project Requirements

- 요구사항

- 본 과제는 EXT2 파일시스템의 내부 구조를 실제 hex data 기반으로 추적하는 Digital Forensic 형태의 실습이다.
- 학생은 RAM disk 위에 생성된 EXT2 파일시스템에서 자신의 학번 끝 세 자리에 따라 할당된 두 개의 target file을 찾아야 한다. 이를 위해 superblock, group descriptor table, inode table, directory entry를 순차적으로 분석하고, target file의 inode number와 실제 data block 위치를 계산해야 한다.
- 또한 direct pointer와 single indirect pointer 구조를 분석하여, target file의 데이터가 어떤 disk block에 저장되어 있는지 xxd 결과를 통해 검증해야 한다.

3. Report Guidelines

■ 보고서 내용

- File System Analysis (필수)
 - 보고서에 각 파일을 찾아가기 까지의 모든 과정을 상세히 설명
 - 본 자료처럼 실행 캡처본 및 명령어와 함께 구체적이고 상세하게 서술할 것
- Bonus (선택)
 - 어느 소스코드의 어느 부분을 수정했는지 **코드의 위치**와 **해당 소스코드 캡처** 및 **이유 설명**
 - 실행 결과 캡처본
- Discussion
 - 기타 내용 자유롭게 작성
 - 새롭게 배웠던 점, 어려웠던 점, 더 공부하고 싶은 점 등
- **생성형 AI (Gemini, ChatGPT) 캡처 또는 그대로 복붙의 경우, 보고서 점수 0점 처리**

4. Report Submission

- 양식

- 제목: os_lab3_분반_학번_이름.pdf
 - Ex) os_lab3_2_32222797_위다연.pdf
 - 구글폼 제출 시 파일명 뒤에 본인의 구글 닉네임이 추가됨 (감점 X)
 - 구글폼 재제출 가능 (가장 마지막에 제출한 제출물을 기준으로 채점)
- 코드 및 터미널 화면 첨부 시 **흰색 바탕으로 캡처**
 - 그렇지 않을 시, 감점의 요인이 됨
 - VS code 사용자: [VS Code 테마 변경 방법](#)
 - Linux 터미널: [리눅스 터미널 색상 변경 방법](#)

4. Grading Criteria

- 총점 100점 (자세한 배점은 하단 표 참고)

구분	세부사항	점수
File System Analysis	Target File 1	30
	Target File 2	40
Bonus	File System Modification	20
Discussion	-	7
Format	-	3

- 유의사항

- Bonus 과제 수행 시, 하나의 보고서에 File System Analysis 와 Bonus 내용을 모두 작성하여 파일 하나만 제출
- 단, 섹션을 구분하여 작성할 것
- 지각 제출 시, 하루에 30% 감점
- 형식 미 준수 시 → 형식 점수 0점

4. Submission Guideline

기한: 2026.06.10 (수) 23:59까지

■ 제출

- 1분반: <https://docs.google.com/forms/d/e/1FAIpQLSfYZPIiQ1ndwxkQIWtCh0eoAj1UOVymMjoCuJEeR0QZB6NZzA/viewform?usp=header>
- 2분반: https://docs.google.com/forms/d/e/1FAIpQLScNt_HAp-2n1N63_HnFeUheXR65QXF5RrWuMeiIhfBF1E6UJg/viewform?usp=header
- 3분반: <https://docs.google.com/forms/d/e/1FAIpQLScM6cp3Mw2aZ-oxinrjoSvI0gUOdbdbYmuMGTVLUBCHRQoesw/viewform?usp=header>
- 구글 폼 양식에 맞춰 제출 (재제출 가능, 가장 마지막 제출을 기준으로 채점)
- 본인 분반에 맞게 제출, 그렇지 않을 시 미제출로 간주함
- 이메일로 제출 금지, 이메일로 제출 시 회신 없이 미제출로 간주함
- 구글 폼 제출 시 본인의 메일로 사본을 받을 수 있음
- **질문이 있을 시 조교의 이메일 또는 git issue에 업로드**
- **메일 작성 시, 본인의 분반과 이름 작성 바랍니다.**

Thank You

2026.05.27

T.A. 위다연

weedayeon@dankook.ac.kr